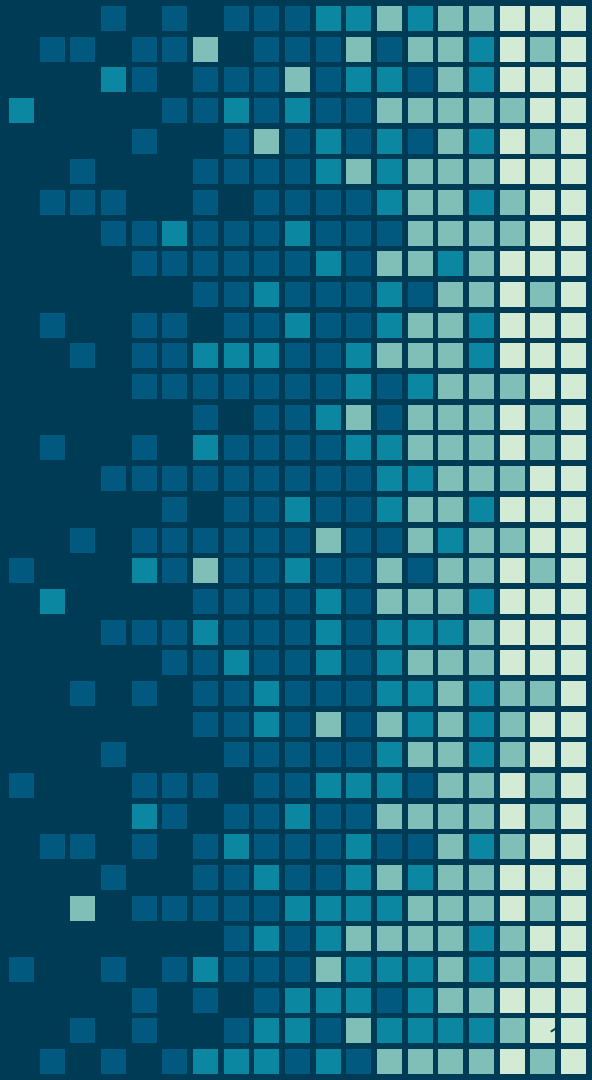
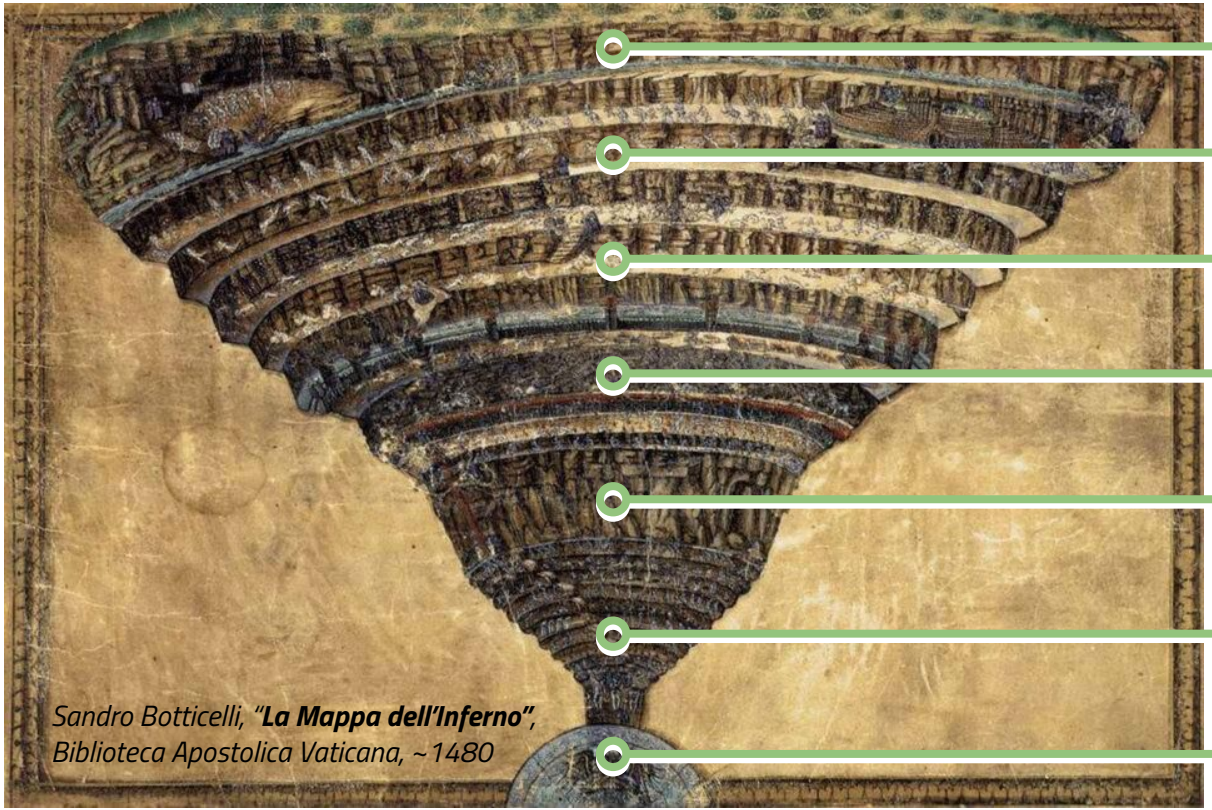
EMBEDDED CONTROL

Prof. Luca De Cicco
luca.decicco@poliba.it
https://c3lab.poliba.it/LDC



PROGRAMMING GPIOs

Where are we?



*Sandro Botticelli, "**La Mappa dell'Inferno**", Biblioteca Apostolica Vaticana, ~1480*

RTOS

Programming with HAL

Programming with CMSIS

Programming registers

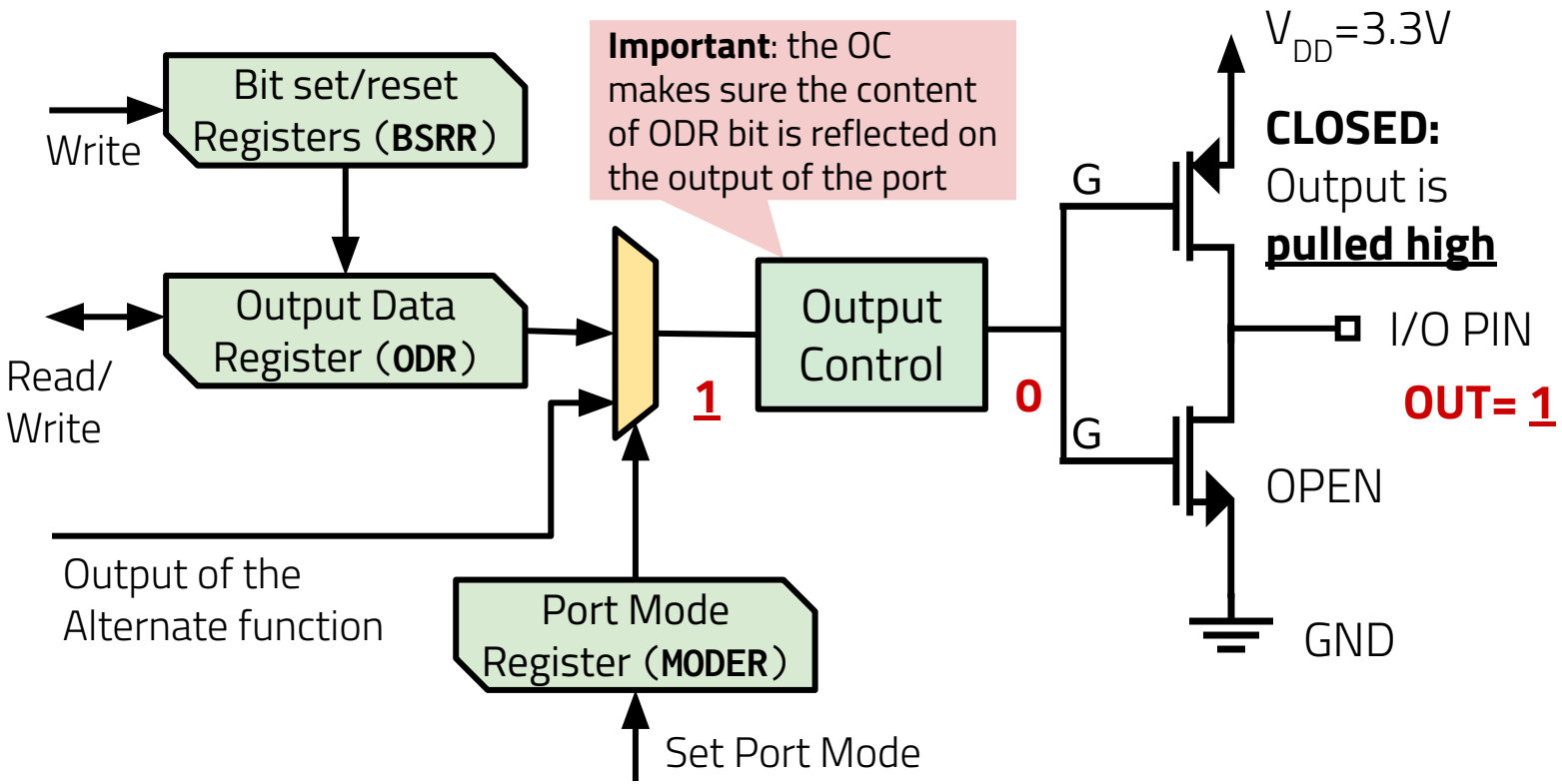
Peripherals Hw

MCU Hw

ARM Cortex-M

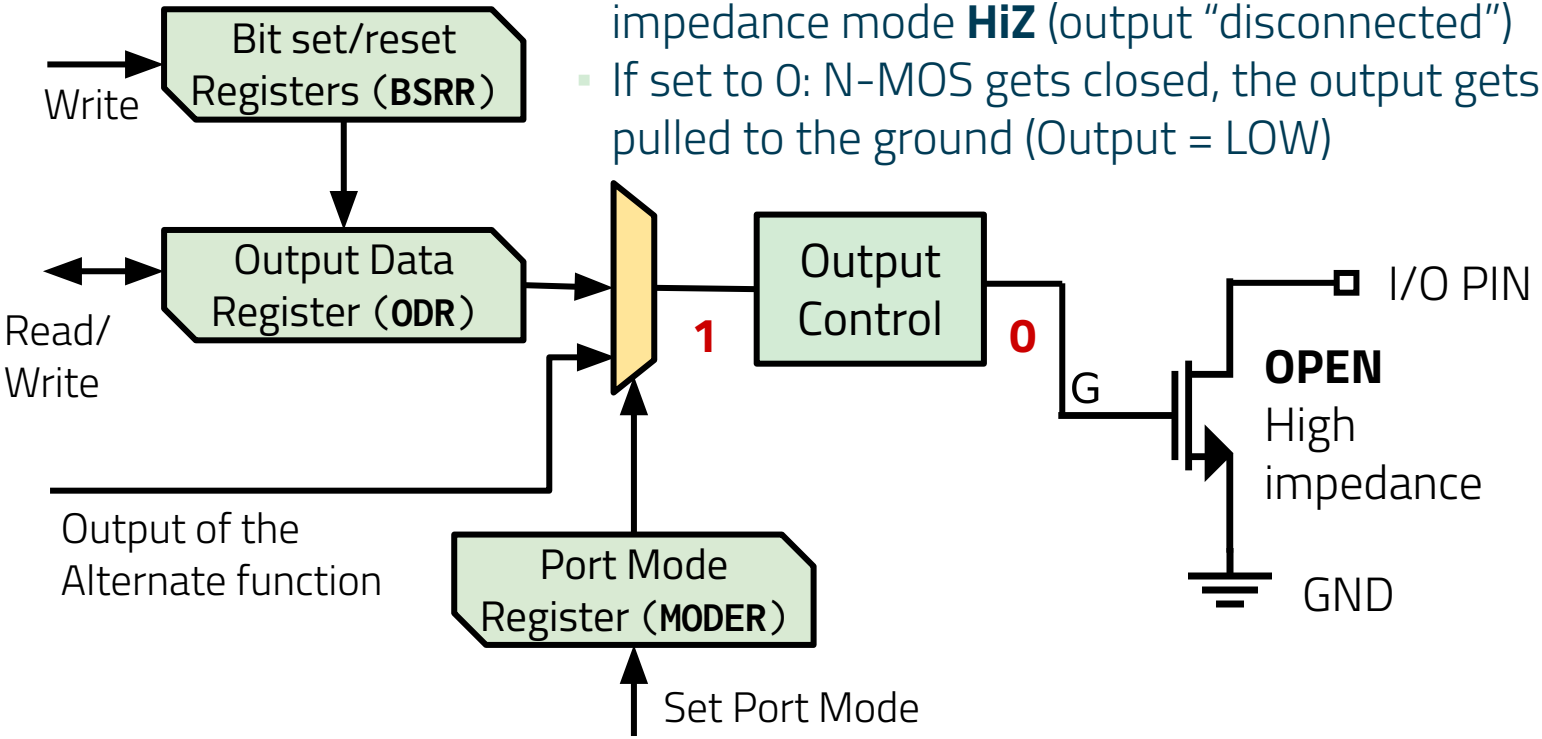
PATH TO ~~REDEMPTION~~ ENLIGHTENMENT

Simplified MCU **Output** Stage Push/Pull



Simplified MCU **Output** Stage **Open Drain**

- If set 1: N-MOS is open, the output is in high impedance mode **HiZ** (output "disconnected")
- If set to 0: N-MOS gets closed, the output gets pulled to the ground (Output = LOW)



GPIO programming introduction

- GPIOs can be programmed through memory mapped registers
- Each GPIO port has a defined memory address base (`GPIOX_BASE`)
- `GPIOA` registers start at `GPIOA_BASE=0x40020000` address (in peripherals address space!)
- Each GPIO port has 1024 Byte address space reserved for its registers (not all the space is used)
- For instance: `GPIOB_BASE = GPIOA_BASE + 0x0400`

GPIO registers: where are them?

0x4002 1C00 - 0x4002 1FFF	GPIOH	AHB1
0x4002 1800 - 0x4002 1BFF	GPIOG	
0x4002 1400 - 0x4002 17FF	GPIOF	
0x4002 1000 - 0x4002 13FF	GPIOE	
0x4002 0C00 - 0x4002 0FFF	GPIOD	
0x4002 0800 - 0x4002 0BFF	GPIOC	
0x4002 0400 - 0x4002 07FF	GPIOB	
0x4002 0000 - 0x4002 03FF	GPIOA	

Notice: all GPIO ports are connected to the AHB1 bus in the STM32446xx MCU

Base addresses for GPIO ports. Each port has 1024 byte of memory address space reserved for its registers (`0x400 = 1024`)

GPIO port registers

```
typedef struct {
  volatile uint32_t MODER;    /* Mode register                - offs: 0x00 */
  volatile uint32_t OTYPER;   /* Output type register         - offs: 0x04 */
  volatile uint32_t OSPEEDR;  /* Output speed register        - offs: 0x08 */
  volatile uint32_t PUPDR;    /* Pull-up/pull-down register   - offs: 0x0C */
  volatile uint32_t IDR;      /* Input data register          - offs: 0x10 */
  volatile uint32_t ODR;      /* Output data register         - offs: 0x14 */
  volatile uint32_t BSRR;     /* Bit set/reset register       - offs: 0x18 */
  volatile uint32_t LCKR;     /* Configuration lock register  - offs: 0x1C */
  volatile uint32_t AFR[2];   /* Alternate function registers - offs: 0x20 */
  volatile uint32_t BRR;      /* Bit Reset register           - offs: 0x28 */
  volatile uint32_t ASCR;     /* Analog switch control regist - offs: 0x2C */
} GPIO_TypeDef;

/* Casting memory address to a pointer */
#define GPIOA ((GPIO_TypeDef *) 0x40020000)
/* F.i., GPIOA->MODER to access the mode register */
```

GPIO Configuration: **GPIOX_MODER** reg

- Recall `X=A,B,C,...,H` port identifier
- A GPIO pin can be configured in different ways according specific bits of a register associated to the correct GPIO port
- Each pin has 2 bits associated in the `GPIOX_MODER` register which set the pin mode:
  - `00`: Digital Input Mode
  - `01`: Digital Output Mode
  - `10`: Alternative Function (AF) Mode
  - `11`: Analog mode (default)

Example: setting **GPIOX_MODER** reg

- Example: set pin PA4 in digital output and PA6 in AF mode
- GPIOX_MODER register bits definition (sect. 7.4.1 ref man):

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

- Need to set:
  - bits 8 and 9 to `01`
  - bits 12 and 13 to `10`

Mock implementation

```
#define GPIO_DIN  0 /*These are used to increase readability */
#define GPIO_DOUT 1
#define GPIO_AF   2
#define GPIO_AN   3
uint32_t set_moder(uint32_t moder, uint8_t pin, uint8_t mode)
{
  const uint32_t moder_msk = 3<<(pin*2);
  moder = (moder & ~moder_msk) | ( mode<<(pin*2) & moder_msk);
  return moder;
}
…
GPIOA->MODER = set_moder(GPIOA->MODER, 4, GPIO_DOUT);
GPIOA->MODER = set_moder(GPIOA->MODER, 6, GPIO_AF);
```

Why do we need two bitwise operations to set only specific bits?

GPIO Configuration: **GPIOX_OTYPER** reg

- The `GPIOX_OTYPER` register decides the "type" of an output GPIO pin, i.e. push-pull or open-drain
- One bit is reserved for each pin of the X-th port in the `GPIOX_OTYPER` register:
  - 0: Push-pull (default)
  - 1: Open-drain

`GPIOA_OTYPER`

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0

Pin **PA3** is set in open-drain
all other pins push-pull

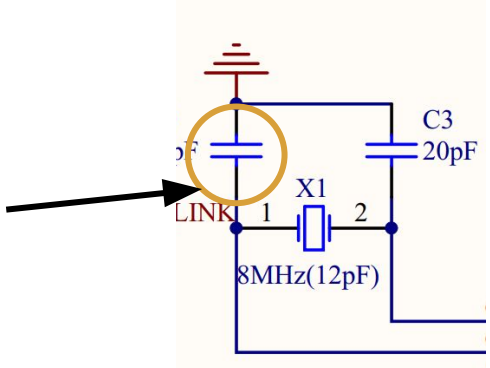
GPIO Configuration: **GPIOX_OSPEEDR** reg

- When used in output mode a GPIO pin "speed" can be set through the `OSPEEDR` register
- It sets the maximum frequency of the digital **output**
- Two bits per pin in the `OSPEEDR` register:
  - `00`: Low-speed
  - `01`: Medium speed
  - `10`: Fast speed
  - `11`: High speed
- Check next slides for actual maximum output frequency

GPIO Configuration: **GPIOX_OSPEEDR** reg

OSPEEDR y[1:0] bit value(1)	Symbol	Parameter	Conditions	Min	Typ	Max	Unit
00	$f_{max(IO)out}$	Maximum frequency(3)	C_L = 50 pF, V_{DD} ≥ 2.7 V	-	-	4	MHz
			C_L = 50 pF, V_{DD} ≥ 1.7 V	-	-	2	
			C_L = 10 pF, V_{DD} ≥ 2.7 V	-	-	8	
			C_L = 10 pF, V_{DD} ≥ 1.8 V	-	-	4	
			C_L = 10 pF, V_{DD} ≥ 1.7 V	-	-	3	

C_L is the capacitor used in the 8MHz external oscillator circuit



GPIO typical “speed” function of **OSPEEDR**

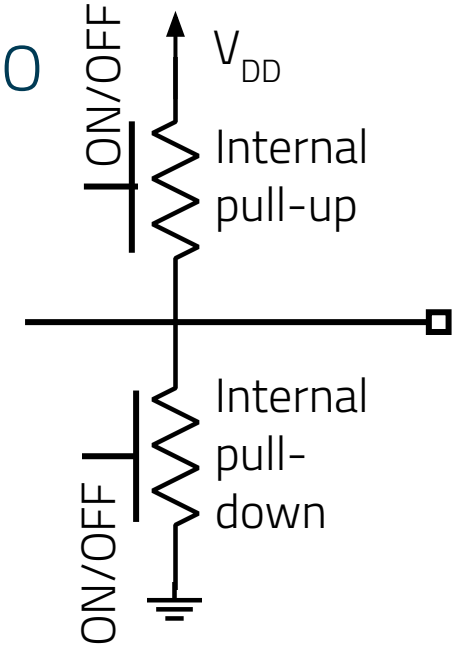
These values are specific to the STM32F446xx MCU with typical values of the capacitor C_L

OSPEEDR	Max Output Frequency (MHz)
00	4 to 8
01	25 to 50
10	50 to 100
11	100 to 180

- Higher OSPEEDR means higher consumption and EMI
- Choose the right setting according to the device you are driving through the PIN

GPIO Configuration: **GPIOX_PUPDR** reg

- This register is used to turn ON/OFF internal pull-up/pull-down of a PIN
- This has an effect only when the GPIO is **configured as input**
- Two bits for each pin of the port:
  - `00`: No pull-up, no pull-down
  - `01`: Pull-up
  - `10`: Pull-down
  - 11: Reserved



GPIO Output Data Register **GPIOX_ODR**

- The **GPIOX_ODR** register sets the output value of the pins of port x
- To turn ON/OFF the y-th pin of port x (i.e. pin Pxy) it is sufficient to set/reset the corresponding bit (the y-th bit) in the GPIOX_ODR register

GPIOA_ODR

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	1	0	1	0	0	0	1	0	0	0

All pins of port A are low, except for pins PA3, PA7, PA9

GPIO port bit set/reset register **GPIOX_BSRR**

- The **GPIOX_BSRR** is used to **atomically** set (turn ON) or reset (turn OFF) the y-th pin of port x.
- The register is divided in two 16bit parts:
- Bits 0 to 15 are used to set the corresponding bit of the ODR
- Bits 16 to 31 are used to reset pins of the ODR
- **Notice**: this register is **write only:** reading the register returns `0x00000000` regardless of the
- **Notice**: writing a `0` in any of the bits of the register does not change the state of the output port!

`GPIOX_BSRR` example

- We want to set pin PA**5** and reset pin PA**8**
- Say the ODR is set as:

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	0	1	1	1	0	0	0	1	0	0	0

- `GPIOA->BSRR = (1 << 5) | (1 << (16 + 8))`
- This is the content of ODR after setting BSRR

15	14	13	12	11	10	9	**8**	7	6	**5**	4	3	2	1	0
0	0	0	0	0	0	1	**0**	1	0	**1**	0	1	0	0	0

GPIO Input Data Register **GPIOX_IDR**

- The **GPIOX_IDR** register stores the digital value for all the pins of port x
- The GPIOx_IDR captures the logical value of the I/O pin at **every AHB1 clock cycle**.
- To read y-th pin of port x (i.e. pin Pxy) it is sufficient to check the corresponding bit (the y-th bit) in the GPIOX_IDR register

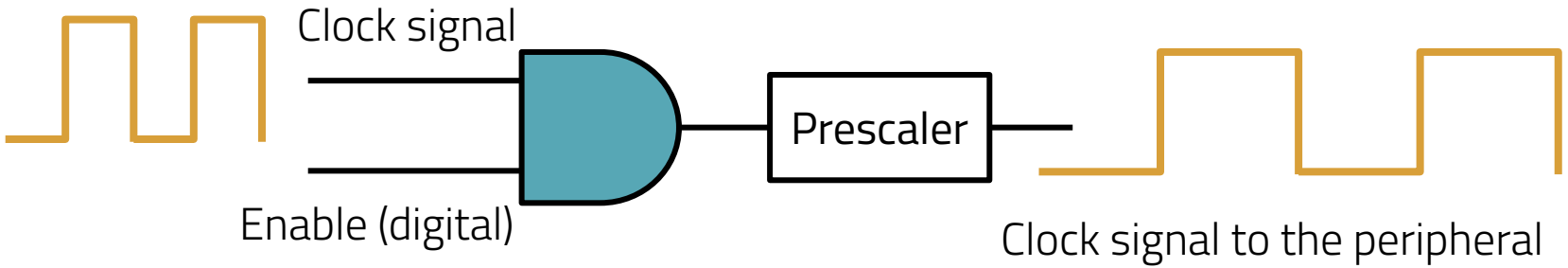
GPIOA_IDR

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	1	1	0	0	0	0	0	0	0	0	0	0

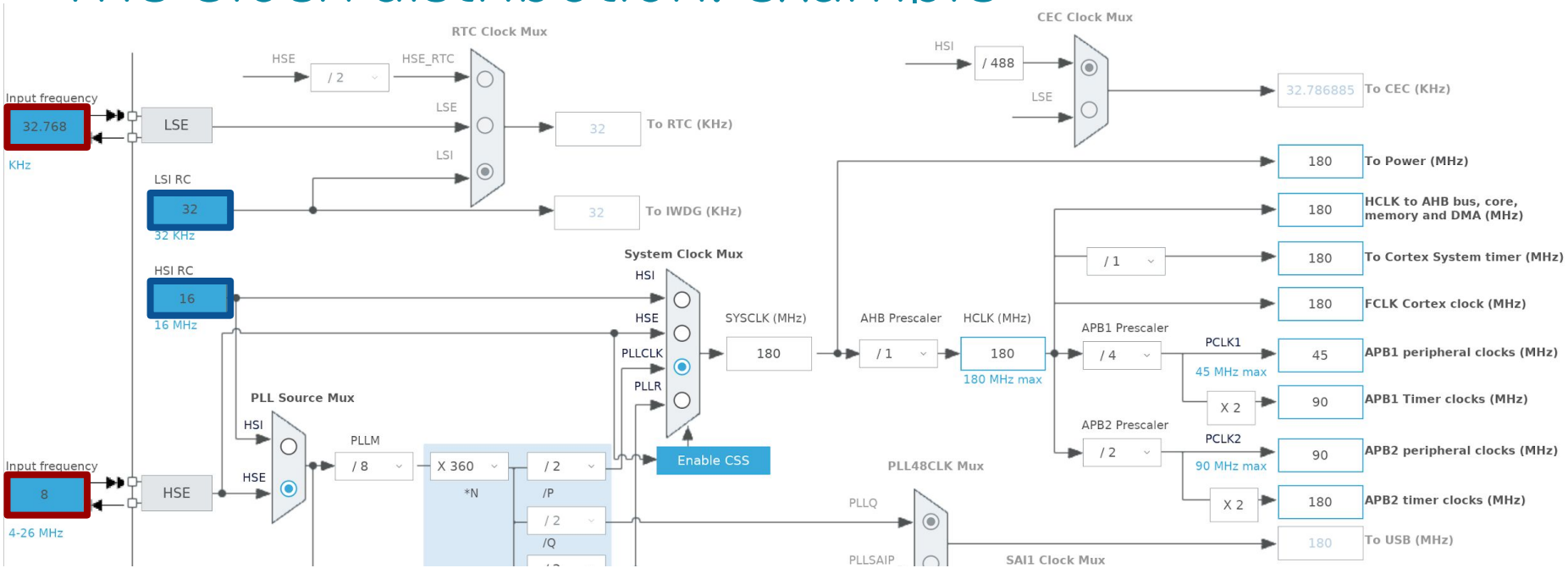
All pins of port A are LOW, except for pins PA10, PA11, PA12

The CPU/MCU **heartbeat**: Clocks

- Clocks give the pace of each operation on CPUs/MCUs
- (Different) Clocks are distributed to buses (peripheral, system,...)
- Clocks are used by each component of the MCU to synchronize them and make the whole system work as expected
- To save energy, clocks are not routed by default to all peripherals
- They must be enabled (by changing bits in a register) through an `Enable` digital signal

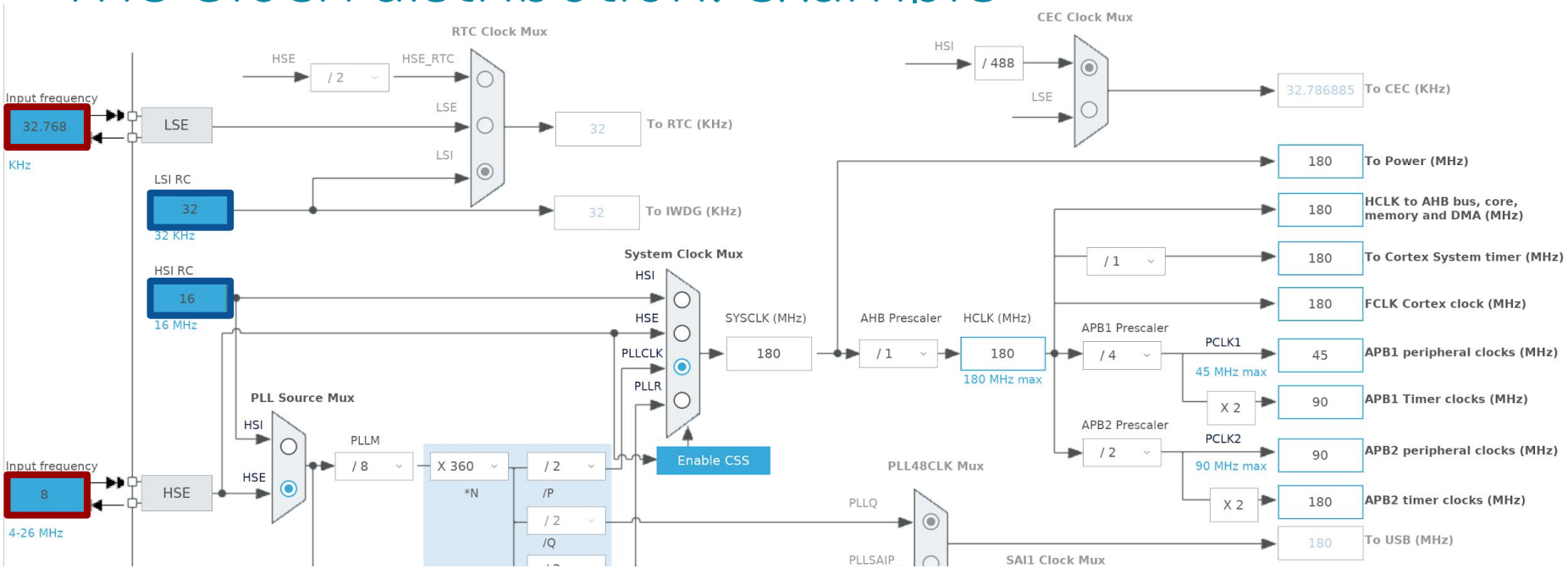


The Clock distribution: example



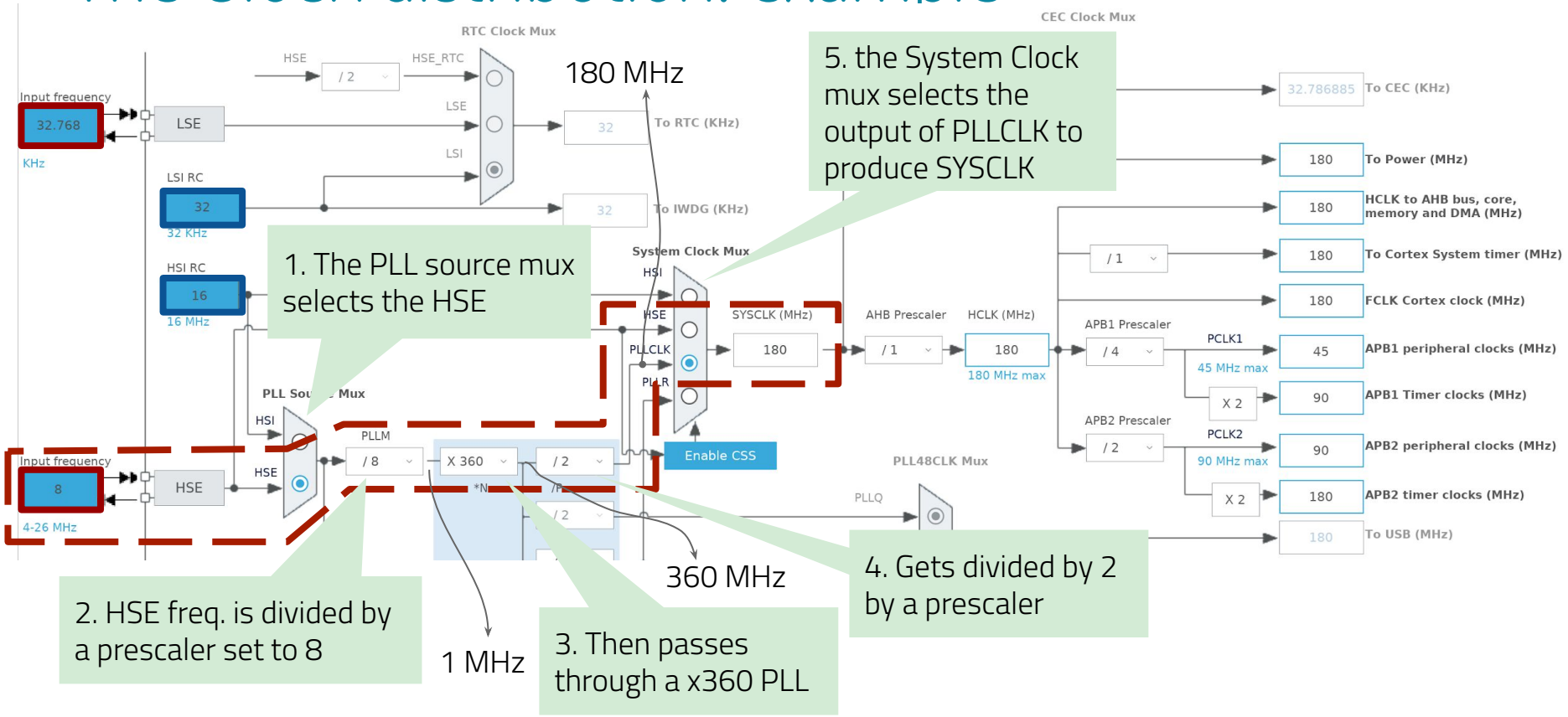
- Two internal oscillators (always present): Low Speed Internal (LSI), High Speed Internal (HSI)
- Two external oscillators (optional): Low Speed External (LSE), High Speed External (HSE)
- Preferable to use external oscillator if present, due to higher accuracy

The Clock distribution: example

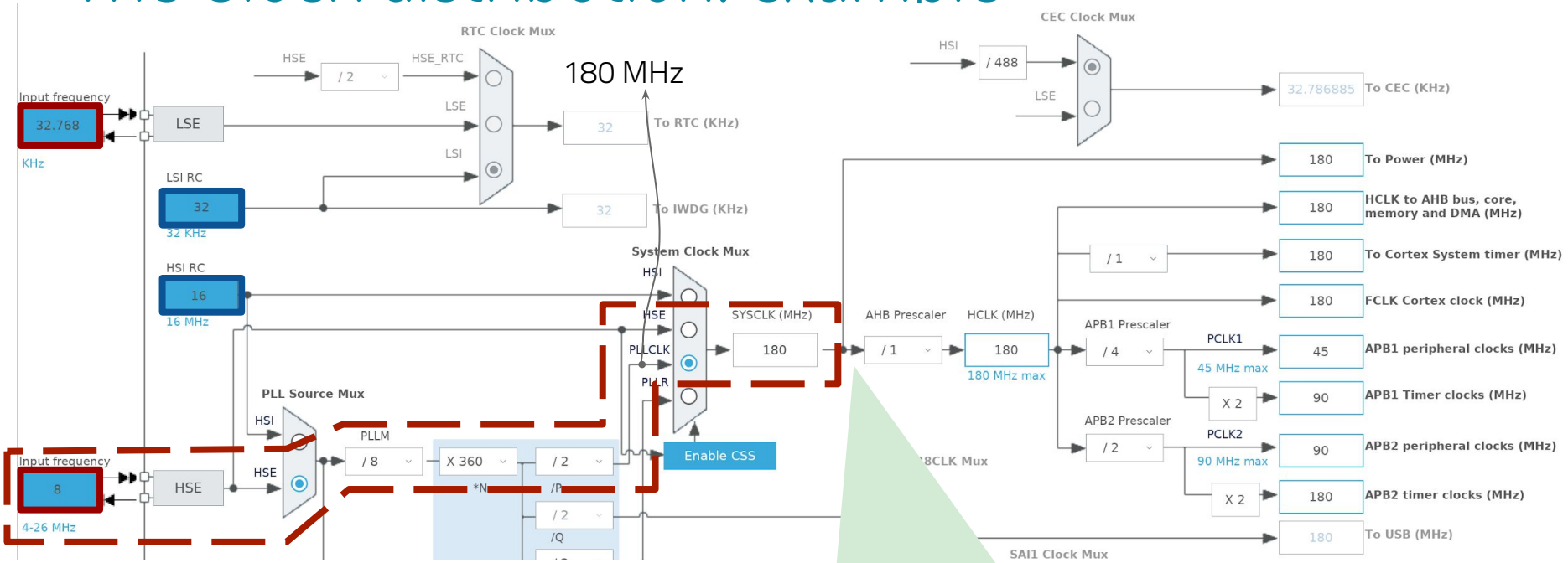


- It is possible to programmatically route different sources to buses possibly by multiplying (PLL) and dividing (prescalers) the clock source frequency
- Muxes are used to select which source to use

The Clock distribution: example



The Clock distribution: example



IMPORTANT: Different buses have different maximum clock frequency. Example: APB1 clock is maximum 45MHz

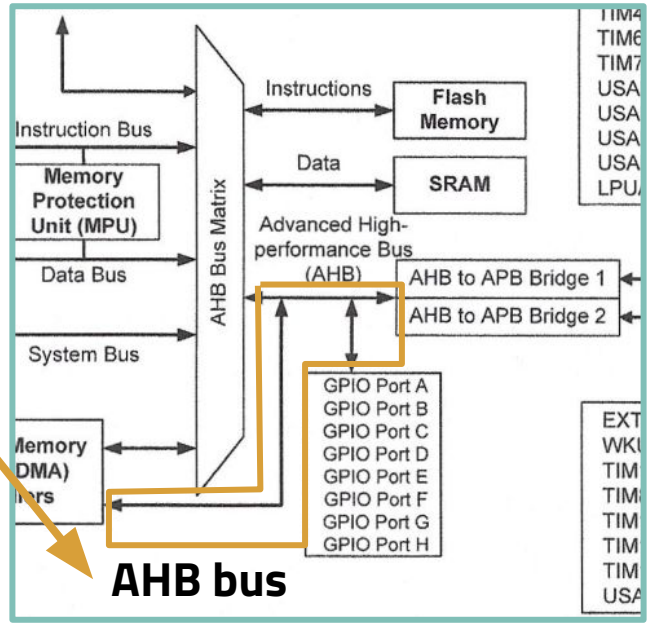
The SYSCLK is routed to different buses possibly prescaled. Notice if AHB prescaler is used it prescales ALL the buses clock frequency

Enable GPIO port clock

- First we need to know to which bus GPIO ports are connected

0x4002 1C00 - 0x4002 1FFF	GPIOH	AHB1
0x4002 1800 - 0x4002 1BFF	GPIOG	
0x4002 1400 - 0x4002 17FF	GPIOF	
0x4002 1000 - 0x4002 13FF	GPIOE	
0x4002 0C00 - 0x4002 0FFF	GPIOD	
0x4002 0800 - 0x4002 0BFF	GPIOC	
0x4002 0400 - 0x4002 07FF	GPIOB	
0x4002 0000 - 0x4002 03FF	GPIOA	

All GPIO ports are connected to the AHB1 bus in the STM32446xx MCU



RCC: Enable GPIO port clock

- The **Reset and clock control** (RCC) is the hardware component controlling clocks and reset operations
- Among all its registers (see RM, from pg. 116-175) we are interested in those used to ENABLE peripheral clocks
- Each bus has its register called **RCC_[BUSNAME]ENR**
- Since we are interested in AHB1 bus we need to check **RCC_AHB1ENR**

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	OTGHS ULPIEN	OTGHS EN	Res.	Res.	Res.	Res.	Res.	Res.	DMA2 EN	DMA1 EN	Res.	Res.	BKP SRAMEN	Res.	Res.
	rw	rw							rw	rw			rw		

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	CRC EN	Res.	Res.	Res.	Res.	GPIOH EN	GPIOG EN	GPIOF EN	GPIOE EN	GPIOD EN	GPIOC EN	GPIOB EN	GPIOA EN
			rw					rw	rw	rw	rw	rw	rw	rw	rw

Accessing to RCC programmatically

```
typedef struct
{
  __IO uint32_t CR;
  __IO uint32_t PLLCFGR;
...
  __IO uint32_t AHB1ENR;       /* AHB1 peripheral clock register off: 0x30 */
  __IO uint32_t AHB2ENR;
...
} RCC_TypeDef;

/* We can use this pointer: RCC->AHB1ENR */
#define RCC  ((RCC_TypeDef *) RCC_BASE)
```

Checklist to program a GPIO

- Check the port/pin of the GPIO
- Enable the clock of the corresponding GPIO port through the **AHB1ENR** register of the RCC
- Select the mode of operation of the GPIO pin using the **MODER** register (dig input, dig output, AF mode, analog)
- Select output type (if output) through **OTYPER** register (push-pull or open drain)
- Select output speed (if output) through the **OSPEEDR** register (low, medium, fast, high)
- Select pull-up/pull-down through **PUPDR** register (no pull-up/pull-down, pull-up, pull-down)

Use the GPIO as a digital I/O

- Once configured the GPIO pin as a digital input or output you can either:
  - If configured as input: read the logical value sensed on the pin by reading the corresponding **`IDR`** register bit
  - If configured as output:change the desired logical value to the pin by writing the corresponding **ODR** register bit
- **Example**: *read the value of pin PB14*.
  You need to check the value of bit n.14 of the `IDR` register associated to GPIO port B

Copyright notice

Copyright © by **Luca De Cicco**, 2019-2026

All rights reserved. No part of the material covered in this slides may be reproduced, stored in a retrieval system, or transmitted, recorder, or otherwise, without prior written permission of the author.